

FASE 1: DIAGNÓSTICO SOLID Y MÉTRICAS BASELINE (CÓDIGO LEGADO)

Alexsander Gonzalez Farrrayans

1.1 Matriz de Diagnóstico de Violaciones SOLID

El análisis estático del ensamblado legado `DentaCare.Legacy` revela una degradación arquitectónica severa centrada en la clase `GestorCitasOdontologicas`, la cual opera como un antipatrón God Class / Blob, asumiendo orquestación de negocio, lógica tarifaria, control de estado de entidades, persistencia relacional acoplada e infraestructura de comunicaciones.

A continuación, se detalla la matriz de incumplimiento a los cinco principios SOLID:

Principio SOLID	Elemento / Método Afectado	Ubicación Exacta	Explicación Técnica del Defecto y Riesgo Arquitectónico
SRP (Single Responsibility Principle)	Clase <code>GestorCitasOdontologicas</code>	<code>GestorCitasOdontologicas.cs</code> Líneas 5–85	La clase posee múltiples ejes de cambio: cálculo financiero de copagos, reglas de penalización por tiempo, control de disponibilidad médica, persistencia directa en base de datos (<code>SqlServerEjecutor</code>) y despacho de mensajería (<code>NotificacionServicio</code>). Una alteración en el esquema SQL o en el proveedor SMTP obliga a recompilar y revalidar las

			reglas de negocio odontológicas.
SRP (<i>Single Responsibility Principle</i>)	Clase <code>Cita</code> (Método de formato)	<code>Cita.cs</code> Líneas 14–16	La entidad de dominio <code>Cita</code> incluye <code>GenerarTextoConfirmacion()</code> , asumiendo la responsabilidad de formateo de presentación y serialización textual para canales externos, mezclando el estado del modelo con la capa de presentación.
OCP (<i>Open/Closed Principle</i>)	<code>AgendarCita</code> (Cálculo de especialidad y convenio)	<code>GestorCitasOdontologicas.cs</code> Líneas 20–47	Violación por encadenamiento condicional (<code>if/else</code>) sobre enteros primitivos. La inclusión de una nueva especialidad (ej. <i>Periodoncia</i>) o un nuevo convenio (ej. <i>Seguro Internacional</i>) exige modificar el cuerpo interno del método, alterando código previamente probado y aumentando el riesgo de regresiones. No está abierto a la extensión ni cerrado a la modificación.
OCP (<i>Open/Closed Principle</i>)	<code>CancelarCita</code> (Penalizaciones dinámicas)	<code>GestorCitasOdontologicas.cs</code> Líneas 68–74	Reglas de penalización acopladas mediante branching condicional a la especialidad fija <code>Cirugia</code> (<code>EspecialidadId == 3</code>). Si la clínica define una política de penalización para <i>Endodoncia</i> , se debe volver a alterar el método.

LSP (<i>Liskov Substitution Principle</i>)	Modelado de Actores (Paciente, Odontologo)	Paciente.cs / Odontologo.cs Uso en GestorCitasOdontologicas.cs	Ausencia de jerarquías polimórficas y modelado anémico. El comportamiento depende del chequeo de flags y tipos primitivos (TipoConvenio, EsPrimeraVez, EspecialidadId). La imposibilidad de sustitución polimórfica fuerza al consumidor a conocer los estados internos de las dependencias.
ISP (<i>Interface Segregation Principle</i>)	NotificacionServicio	NotificacionServicio.cs Líneas 9–21	La clase monolítica acopla correo y mensajería de texto en una única operación: EnviarEmailYSms(Cita, string). Los clientes que únicamente requieren notificar vía SMS o vía Correo electrónico están forzados a depender de la infraestructura de ambos canales simultáneamente (cliente SMTP y API key de Twilio).
DIP (<i>Dependency Inversion Principle</i>)	Instanciación directa en GestorCitasOdontologicas	GestorCitasOdontologicas.cs Líneas 7–8	El módulo de alto nivel (GestorCitasOdontologicas) depende directamente de implementaciones concretas de bajo nivel: new SqlServerEjecutor() y new NotificacionServicio(). Es imposible realizar pruebas unitarias aisladas sin levantar una base de datos SQL Server activa y un servidor SMTP local.

DIP (<i>Dependency Inversion Principle</i>)	Infraestructura Concreta	SqlServerEjecutor.cs Líneas 10, 14, 27	La persistencia implementa cadenas de conexión hardcodeadas (Server=sql.dentacare.com;...) e interactúa con el driver concreto ADO.NET (SqlConnection, SqlCommand), violando la inversión de control.
--	--------------------------	---	---

1.2 Línea Base Cuantitativa (Baseline) de GestorCitasOdontologicas

Para establecer el estado de la arquitectura legada, se calculan las métricas formales de la suite de Robert C. Martin y Chidamber & Kemerer sobre la clase central.

```

+-----+
-----+
|                                     GESTORCITASODONTOLOGICAS  (LEGACY)
|
+-----+
-----+
| - _db: SqlServerEjecutor
|
| - _notificador: NotificacionServicio
|
| - _totalRecaudadoMes: decimal
|
| - _totalCitasCanceladas: int
|
+-----+
-----+
| + AgendarCita(paciente, odontologo, fechaHora, tipoEspecialidad,
radiografia): Cita
| + CancelarCita(cita, fechaHoraCancelacion): decimal
|
| + ObtenerTotalRecaudado(): decimal
|
| + ObtenerTotalCanceladas(): int
|
+-----+
-----+

```

1.2.1 Cohesión: Cálculo Matemático Paso a Paso de LCOM96b

La métrica LCOM96b (Henderson-Sellers, 1996) cuantifica la falta de cohesión en una clase en el intervalo $[0, 1]$. Un valor cercano a 0 representa alta cohesión (diseño óptimo), mientras que un valor cercano a 1 indica una fragmentación severa de responsabilidades.

Fórmula Formal:

$$LCOM96b = \frac{\frac{1}{a} \sum_{j=1}^a [m - \mu(A_j)]}{m-1} \equiv \frac{1}{a} \sum_{j=1}^a \left[\frac{m - \mu(A_j)}{m} \right] (\text{versión normalizada sobre } m)$$

Adoptando la formulación estricta de Henderson-Sellers para $m > 1$:

$$LCOM96b = \frac{\left(\frac{1}{a} \sum_{j=1}^a (m - \mu(A_j)) \right)}{m-1} = \frac{m - \frac{1}{a} \sum_{j=1}^a \mu(A_j)}{m-1}$$

Parámetros del Sistema Legado:

- Métodos analizados ($m = 4$):
 - i. M_1 : AgendarCita
 - ii. M_2 : CancelarCita
 - iii. M_3 : ObtenerTotalRecaudado
 - iv. M_4 : ObtenerTotalCanceladas

- Atributos de instancia analizados ($a = 4$):
 - i. A_1 : _db
 - ii. A_2 : _notificador
 - iii. A_3 : _totalRecaudadoMes
 - iv. A_4 : _totalCitasCanceladas

Matriz de Acceso Atributo-Método ($\mu(A_j)$):

Atributo (A_j)	M_1 (Agen dar)	M_2 (Cance lar)	M_3 (TotalReca udado)	M_4 (TotalCance ladas)	$\mu(A_j)$ (Total métodos que acceden)
A_1 : _db	X	X	—	—	2
A_2 : _notificador	X	X	—	—	2
A_3 : _totalRecaudadoMes	X	—	X	—	2
A_4 : _totalCitasCancela das	—	X	—	X	2

Desarrollo Matemático:

1. Calculamos el promedio de métodos que acceden a cada atributo:

$$2. \bar{\mu} = \frac{1}{a} \sum_{j=1}^4 \mu(A_j) = \frac{2+2+2+2}{4} = \frac{8}{4} = 2.0$$

3. Aplicamos la fórmula de normalización requerida en la guía:

$$4. LCOM96b = \frac{1}{a} \sum_{j=1}^a \left[\frac{m - \mu(A_j)}{m} \right]$$

5.

$$6. LCOM96b = \frac{1}{4} \left[\left(\frac{4-2}{4} \right) + \left(\frac{4-2}{4} \right) + \left(\frac{4-2}{4} \right) + \left(\frac{4-2}{4} \right) \right]$$

7.

$$8. LCOM96b = \frac{1}{4} \left[\frac{2}{4} + \frac{2}{4} + \frac{2}{4} + \frac{2}{4} \right] = \frac{1}{4} [0.5 + 0.5 + 0.5 + 0.5] = \frac{1}{4} [2.0] = 0.50$$

Si se aplica la formulación de escala Henderson-Sellers estándar:

$$LCOM96b_{H-S} = \frac{m - \bar{\mu}}{m-1} = \frac{4-2.0}{4-1} = \frac{2.0}{3} \approx 0.667$$

Diagnóstico: Un LCOM de 0.50 (ó 0.67) confirma una pobre cohesión interna. La clase contiene dos sub-componentes independientes que no comparten estado: el subsistema transaccional (_db, _notificador) y el subsistema de acumulación de métricas en memoria (_totalRecaudadoMes, _totalCitasCanceladas).

1.2.2 Acoplamiento Eferente (C_e) y Aferente (C_a)

Acoplamiento Eferente (C_e - Dependencias Salientes):

Número de tipos fuera de la clase de los cuales `GestorCitasOdontologicas` depende directamente:

1. `SqlServerEjecutor` (Instanciación directa en campo)
2. `NotificacionServicio` (Instanciación directa en campo)
3. `Paciente` (Parámetro en `AgendarCita`)
4. `Odontologo` (Parámetro e inspección de estado)
5. `Cita` (Instanciación y retorno)
6. `DateTime` (Tipo de valor del framework)
7. `TimeSpan` (Tipo de cálculo de tiempo)
8. `Guid` (Generación de identificadores)
9. `Exception` (Manejo de flujo anómalo)

Excluyendo los tipos base del BCL de C# (`DateTime`, `TimeSpan`, `Guid`, `Exception`), el acoplamiento eferente hacia tipos de negocio e infraestructura es: $C_e = 5$

`\quad \text{({SqlServerEjecutor, NotificacionServicio, Paciente, Odontologo, Cita})}`

Acoplamiento Aferente (C_a - Dependencias Entrantes):

Número de tipos externos que dependen de `GestorCitasOdontologicas`:

1. `Program` (Consumidor directo en consola)
2. $C_a = 1$

1.2.3 Inestabilidad (I), Abstracción (A) y Distancia a la Secuencia Principal (D)

Inestabilidad (I):

Mide la resistencia al cambio de la clase.

$$I = \frac{C_e}{C_e + C_a} = \frac{5}{5+1} = \frac{5}{6} \approx 0.833$$

Interpretación: La clase es altamente inestable ($I \rightarrow 1$). Depende de muchas clases concretas y cualquier variación en la infraestructura la fuerza a cambiar.

Abstracción (A):

Relación entre elementos abstractos y el total de tipos.

$$A = \frac{\Sigma m_a}{\Sigma m_c} = \frac{0}{1} = 0.0$$

Donde $\Sigma m_a = 0$ (no implementa interfaces ni contiene métodos abstractos) y

$\Sigma m_c = 1$ (es puramente concreta).

Distancia Normalizada a la Secuencia Principal (D):

Mide el equilibrio entre abstracción y acoplamiento:

$$D = |A + I - 1| = |0.0 + 0.833 - 1| = |-0.167| = 0.167$$

Análisis del Plano A vs. I y Demostración de la Zona de Dolor:

- Coordenadas de la clase en el plano: $(I, A) = (0.833, 0.0)$.
- Aunque la clase posee alta inestabilidad técnica ($I = 0.833$), arquitectónicamente exhibe una contradicción: actúa como el núcleo orquestador del sistema (responsabilidad crítica), pero su implementación es 100% concreta ($A = 0.0$).
- Riesgo: Si otras clases comienzan a depender de `GestorCitasOdontologicas` (C_a se incrementa), la inestabilidad tenderá a 0:

$$\bullet \lim_{C_a \rightarrow \infty} I = \lim_{C_a \rightarrow \infty} \frac{5}{5+C_a} = 0.0$$

- En ese escenario, con $A = 0.0$ e $I = 0.0$, la clase colapsa de lleno en la Zona de Dolor (Zone of Pain): un componente ríspido, con clientes acoplados

a una implementación concreta sin interfaces, haciendo que cualquier cambio en la base de datos o reglas tarifarias bloquee todo el ciclo de despliegue.

FASE 3: RECÁLCULO DE MÉTRICAS E IMPACTO CUANTITATIVO

3.1 Recálculo de Métricas sobre la Clase Orquestadora Refactorizada (CitaAppService)

```
+-----+
+-----+
|                                     CITAAPPSERVICE (REFACTORED)
|
+-----+
+-----+
| - _citaRepository: ICitaRepository
|
| - _notificador: IServicioNotificacionCitas
|
| - _calculadoraCopago: ICalculadoraCopagoStrategy
|
| - _politicaCancelacion: IPoliticaCancelacionStrategy
|
| - TotalRecaudadoMes: decimal
|
| - TotalCitasCanceladas: int
|
+-----+
+-----+
| + AgendarCita(paciente, odontologo, fechaHora, requiereRadiografia): Cita
|
| + CancelarCita(cita, fechaHoraCancelacion): decimal
|
+-----+
+-----+
```

3.1.1 Desarrollo Matemático LCOM96b de CitaAppService

- Métodos analizados ($m = 2$):
 - i. M_1 : AgendarCita
 - ii. M_2 : CancelarCita
- Atributos de instancia ($a = 6$):
 - i. A_1 : _citaRepository
 - ii. A_2 : _notificador
 - iii. A_3 : _calculadoraCopago
 - iv. A_4 : _politicaCancelacion
 - v. A_5 : TotalRecaudadoMes
 - vi. A_6 : TotalCitasCanceladas

Matriz de Acceso Atributo-Método ($\mu(A_j)$):

Atributo (A_j)	M_1 (AgendarCita)	M_2 (CancelarCita)	$\mu(A_j)$ (Total métodos que acceden)
A_1 : _citaRepository	X	X	2
A_2 : _notificador	X	X	2
A_3 : _calculadoraCopago	X	—	1
A_4 : _politicaCancelacion	—	X	1
A_5 : TotalRecaudadoMes	X	—	1
A_6 : TotalCitasCanceladas	—	X	1

Cálculo LCOM96b Normalizado:

$$LCOM96b = \frac{1}{a} \sum_{j=1}^a \left[\frac{m - \mu(A_j)}{m} \right]$$

$$LCOM96b = \frac{1}{6} \left[\left(\frac{2-2}{2} \right) + \left(\frac{2-2}{2} \right) + \left(\frac{2-1}{2} \right) + \left(\frac{2-1}{2} \right) + \left(\frac{2-1}{2} \right) + \left(\frac{2-1}{2} \right) \right]$$

$$LCOM96b = \frac{1}{6} [0 + 0 + 0.5 + 0.5 + 0.5 + 0.5] = \frac{1}{6} [2.0] = 0.333$$

Fórmula Henderson-Sellers estándar:

$$\bar{\mu} = \frac{2+2+1+1+1+1}{6} = \frac{8}{6} = 1.333$$

$$LCOM96b_{H-S} = \frac{m - \bar{\mu}}{m - 1} = \frac{2 - 1.333}{2 - 1} = 0.667$$

En las clases puras de estrategia de dominio (`CalculadoraCopagoCompuesta` y `PoliticaCancelacionEstandar`), todos sus métodos acceden al 100% de sus atributos de configuración, logrando un $LCOM96b = 0.0$ (cohesión perfecta).

3.1.2 Acoplamiento Eferente (C_e) y Aferente (C_a) de CitaAppService

- Dependencias Salientes (C_e):
 - i. ICitaRepository (Interfaz)
 - ii. IServicioNotificacionCitas (Interfaz)
 - iii. ICalculadoraCopagoStrategy (Interfaz)
 - iv. IPoliticaCancelacionStrategy (Interfaz)
 - v. Paciente (Entidad de dominio)
 - vi. Odontologo (Entidad de dominio)
 - vii. Cita (Entidad de dominio)
 - viii. $C_e = 7$ (4 de las cuales son abstracciones puras)
 - ix.
- Dependencias Entrantes (C_a):
 - i. Program (Capa de Presentación / API)
 - ii. $C_a = 1$

3.1.3 Inestabilidad (I), Abstracción (A) y Distancia (D)

Para el paquete/módulo de Aplicación y Dominio Refactorizado:

- Abstracción (A): En la nueva arquitectura, las dependencias se gestionan mediante contratos abstractos:
 - **Clases concretas:** CitaAppService, Cita, Paciente, Odontologo, CalculadoraCopagoCompuesta, PoliticaCancelacionEstandar ($N_c = 6$).
 - **Abstracciones (Interfaces):** ICitaRepository, IServicioNotificacionCitas, INotificadorEmail, INotificadorSms, ICalculadoraCopagoStrategy, IPoliticaCancelacionStrategy ($N_a = 6$).
 - $$A_{\text{módulo}} = \frac{N_a}{N_a + N_c} = \frac{6}{6+6} = 0.50$$
 -
- Inestabilidad del Módulo (I):
 - $$I = \frac{C_e}{C_e + C_a} = \frac{7}{7+1} = 0.875$$
- Distancia a la Secuencia Principal (D):
 - $$D = |A + I - 1| = |0.50 + 0.875 - 1| = |0.375| = 0.375$$

3.2 Tabla Comparativa "Antes (Legacy) vs. Después (Refactorizado)"

Métrica / Atributo Arquitectónico	Estado Legacy (GestorCitasO dontologicas)	Estado Refactorizado (CitaAppService + Dominio)	Justificación y Beneficio Cualitativo / Cuantitativo
LCOM96b (Cohesión de Métodos)	0.50 (Henderson-Sellers: 0.67)	0.33 (Estrategias: 0.00)	Mejora drástica de cohesión. Se eliminó la persistencia y la mensajería del orquestador; las estrategias encapsulan su propio cálculo.
Acoplamiento Concreto (C_e Concreto)	5 tipos concretos	0 dependencias de infraestructura concreta	Cumplimiento total del DIP: CitaAppService desconoce SqlConnection y SmtplibClient.
Abstracción (A)	0.00	0.50	Transición de una estructura monolítica concreta a una arquitectura desacoplada gobernada por interfaces.
Zona de Dolor (Zone of Pain)	Alto Riesgo ($A = 0$, riesgo de inmovilidad si $C_a \uparrow$)	Completamente Mitigado ($A > 0$)	El diseño admite clientes entrantes sin rigidez, facilitando la sustitución vía mocks/stubs en pruebas unitarias.
Ciclomatic Complexity ($V(G)$) en Tarifas	Alto ($V(G) = 11$, switches y branches anidados)	Bajo ($V(G) = 1$, lookup dictionaries y polimorfismo)	La lógica tarifaria ya no crece en branches condicionales, minimizando la densidad de defectos.
Testabilidad Unitaria	0% (Requiere red, SQL Server y SMTP activos)	100% Aislable	Es posible testear la orquestación en milisegundos mediante stubs en memoria de ICitaRepository.